

Endpoint Data Collection & Observability using OSquery

Installation ✓

```
root@k8s-cp:~# osqueryi -version
osqueryi version 5.23.1
```

Exploring OSquery

Exploring tables

```
root@k8s-cp:~# sudo osqueryi
Using a virtual database. Need help, type '.help'
osquery> .help
Welcome to the osquery shell. Please explore your OS!
You are connected to a transient 'in-memory' virtual database.

.all [TABLE]      Select all from a table
.bail ON|OFF      Stop after hitting an error
.connect PATH     Connect to an osquery extension socket
.disconnect       Disconnect from a connected extension socket
.echo ON|OFF      Turn command echo on or off
.exit            Exit this program
.features         List osquery's features and their statuses
.headers ON|OFF   Turn display of headers on or off
.help            Show this message
.mode MODE       Set output mode where MODE is one of:
                  csv      Comma-separated values
                  column   Left-aligned columns see .width
                  line     One value per line
                  list     Values delimited by .separator string
                  pretty   Pretty printed SQL results (default)
.nullvalue STR   Use STRING in place of NULL values
.print STR...    Print literal STRING
.quit           Exit this program
.schema [TABLE]  Show the CREATE statements
.separator STR   Change separator used by output mode
.socket         Show the local osquery extensions socket path
.show           Show the current values for various settings
.summary        Alias for the show meta command
.tables [TABLE]  List names of tables
.types [SQL]     Show result of getQueryColumns for the given query
.width [NUM1]+   Set column widths for "column" mode
.timer ON|OFF    Turn the CPU timer measurement on or off
osquery>
```

```

root@k8s-cp:~# osqueryi .tables | head -n 35
=> acpi_tables
=> apparmor_events
=> apparmor_profiles
=> apt_sources
=> arp_cache
=> augeas
=> authorized_keys
=> azure_instance_metadata
=> azure_instance_tags
=> block_devices
=> bpf_process_events
=> bpf_socket_events
=> carbon_black_info
=> carves
=> certificates
=> chrome_extension_content_scripts
=> chrome_extensions
=> cpu_info
=> cpu_time
=> cpuid
=> crontab
=> curl
=> curl_certificate
=> deb_package_files
=> deb_packages
=> device_file
=> device_hash
=> device_partitions
=> disk_encryption
=> dns_resolvers
=> docker_container_envs
=> docker_container_fs_changes
=> docker_container_labels
=> docker_container_mounts
=> docker_container_networks

```

Exploring schemas

```

osquery> .schema processes
CREATE TABLE processes('pid' BIGINT, 'name' TEXT, 'path' TEXT, 'cmdline' TEXT, 'state' TEXT, 'cwd' TEXT, 'root' TEXT, 'uid' BIGINT, 'gid' BIGINT, 'euid' BIGINT,
'egid' BIGINT, 'suid' BIGINT, 'sgid' BIGINT, 'on_disk' INTEGER, 'wired_size' BIGINT, 'resident_size' BIGINT, 'total_size' BIGINT, 'user_time' BIGINT, 'system_t
ime' BIGINT, 'disk_bytes_read' BIGINT, 'disk_bytes_written' BIGINT, 'start_time' BIGINT, 'parent' BIGINT, 'pgroup' BIGINT, 'threads' INTEGER, 'nice' INTEGER, 'e
levated_token' INTEGER HIDDEN, 'secure_process' INTEGER HIDDEN, 'protection_type' TEXT HIDDEN, 'virtual_process' INTEGER HIDDEN, 'elapsed_time' BIGINT HIDDEN, 'e
handle_count' BIGINT HIDDEN, 'percent_processor_time' BIGINT HIDDEN, 'upid' BIGINT HIDDEN, 'uppid' BIGINT HIDDEN, 'cpu_type' INTEGER HIDDEN, 'cpu_subtype' INTEG
ER HIDDEN, 'translated' INTEGER HIDDEN, 'cgroup_path' TEXT, PRIMARY KEY ('pid')) WITHOUT ROWID;
osquery> .schema users
CREATE TABLE users('uid' BIGINT, 'gid' BIGINT, 'uid_signed' BIGINT, 'gid_signed' BIGINT, 'username' TEXT, 'description' TEXT, 'directory' TEXT, 'shell' TEXT, 'u
uid' TEXT COLLATE NOCASE, 'type' TEXT HIDDEN, 'is_hidden' INTEGER HIDDEN, 'pid_with_namespace' INTEGER HIDDEN, 'include_remote' INTEGER HIDDEN, PRIMARY KEY ('ui
d', 'username', 'uid', 'pid_with_namespace', 'include_remote')) WITHOUT ROWID;
osquery> .schema process_
process_envs          process_file_events    process_namespaces    process_open_pipes
process_events        process_memory_map      process_open_files     process_open_sockets
osquery> .schema process_open_
process_open_files    process_open_pipes      process_open_sockets
osquery> .schema process_open_sockets
...> ;
Error: near "schema": syntax error
osquery> .schema process_open_sockets
CREATE TABLE process_open_sockets('pid' INTEGER, 'fd' BIGINT, 'socket' BIGINT, 'family' INTEGER, 'protocol' INTEGER, 'local_address' TEXT, 'remote_address' TEXT
, 'local_port' INTEGER, 'remote_port' INTEGER, 'path' TEXT, 'state' TEXT, 'net_namespace' TEXT, PRIMARY KEY ('pid', 'fd', 'socket', 'family', 'protocol', 'local
_address', 'remote_address', 'local_port', 'remote_port', 'path', 'state', 'net_namespace')) WITHOUT ROWID;
osquery> .schema listening_ports
Error: unknown command or invalid arguments: "schema". Enter ".help" for help
osquery> .schema listening_ports
CREATE TABLE listening_ports('pid' INTEGER, 'port' INTEGER, 'protocol' INTEGER, 'family' INTEGER, 'address' TEXT, 'fd' BIGINT, 'socket' BIGINT, 'path' TEXT, 'ne
t_namespace' TEXT);
osquery>

```

Choosing and collecting endpoint observables

```
osquery> SELECT name, version, platform, platform_like FROM os_version;
+-----+-----+-----+-----+
| name   | version                               | platform | platform_like |
+-----+-----+-----+-----+
| Ubuntu | 26.04.1 LTS (Resolute Raccoon)       | ubuntu   | debian         |
+-----+-----+-----+-----+
```

```
osquery> SELECT hostname, cpu_brand, physical_memory FROM system_info;
+-----+-----+-----+
| hostname | cpu_brand                               | physical_memory |
+-----+-----+-----+
| k8s-cp   | 13th Gen Intel(R) Core(TM) i7-13620H | 5620969472      |
+-----+-----+-----+
```

```
osquery> SELECT uid, gid, username, directory, shell FROM users ORDER BY uid;
+-----+-----+-----+-----+-----+
| uid  | gid  | username      | directory      | shell      |
+-----+-----+-----+-----+-----+
| 0    | 0    | root          | /root          | /bin/bash  |
| 1    | 1    | daemon        | /usr/sbin      | /usr/sbin/nologin |
| 2    | 2    | bin           | /bin           | /usr/sbin/nologin |
| 3    | 3    | sys           | /dev           | /usr/sbin/nologin |
| 4    | 65534 | sync          | /bin           | /bin/sync  |
| 5    | 60   | games         | /usr/games     | /usr/sbin/nologin |
| 6    | 12   | man           | /var/cache/man | /usr/sbin/nologin |
| 7    | 7    | lp            | /var/spool/lpd | /usr/sbin/nologin |
| 8    | 8    | mail          | /var/mail      | /usr/sbin/nologin |
| 9    | 9    | news          | /var/spool/news | /usr/sbin/nologin |
| 10   | 10   | uucp          | /var/spool/uucp | /usr/sbin/nologin |
| 13   | 13   | proxy         | /bin           | /usr/sbin/nologin |
| 33   | 33   | www-data      | /var/www       | /usr/sbin/nologin |
| 34   | 34   | backup        | /var/backups   | /usr/sbin/nologin |
| 38   | 38   | list          | /var/list      | /usr/sbin/nologin |
| 39   | 39   | irc           | /run/ircd      | /usr/sbin/nologin |
| 42   | 65534 | _apt          | /nonexistent   | /usr/sbin/nologin |
| 100  | 46   | usbmux        | /var/lib/usbmux | /usr/sbin/nologin |
| 987  | 65534 | sshd          | /run/sshd      | /usr/sbin/nologin |
| 988  | 988  | polkitd       | /              | /usr/sbin/nologin |
| 989  | 989  | _chrony       | /var/lib/chrony | /usr/sbin/nologin |
| 990  | 990  | systemd-resolve | /              | /usr/sbin/nologin |
| 996  | 996  | messagebus    | /nonexistent   | /usr/sbin/nologin |
| 997  | 997  | dhcpcd        | /usr/lib/dhcpcd | /bin/false  |
| 998  | 998  | systemd-network | /              | /usr/sbin/nologin |
| 1000 | 1000 | amin          | /home/amin     | /bin/bash  |
| 65534 | 65534 | nobody        | /nonexistent   | /usr/sbin/nologin |
+-----+-----+-----+-----+-----+
```

⇒ The local account with uid 0 is **root**, its shell is **/bin/bash**

393145	osqueryi	/opt/osquery/bin/osqueryd	3540	0	0	osqueryi
393246	sleep	/snap/microk8s/9072/bin/sleep	356846	0	1	sleep 5
			1110	0	1	

12. Which of these attributes could be useful for detecting suspicious process execution? Briefly explain why?

- ⇒ **Name:** it could be the name of well-known malicious or unneeded process names
- ⇒ **PARENT:** Knowing PP helps in tracing who ran the process, (powershell/cmd can be suspicious when its PP is outlook or something like that)

```

root@k8s-cp: ~
Using a virtual database. Need help, type '.help'
osquery> select name, pid, path, cmdline, uid from processes where path like '%tmp%';
osquery> ^C

amin@k8s-cp:~$ sudo -i
root@k8s-cp:~# osqueryi
Using a virtual database. Need help, type '.help'
osquery> select name, pid, path, cmdline, uid from processes where path like '%tmp%';
osquery> select name, pid, path, cmdline, uid from processes where path like '%/tmp/%';
osquery> select name, pid, path, cmdline, uid from processes where cmdline like '%/tmp/%';
+-----+-----+-----+-----+-----+
| name      | pid   | path           | cmdline                                     | uid   |
+-----+-----+-----+-----+-----+
| testscript | 49940 | /usr/bin/dash  | /bin/sh /tmp/testscript 3600 | 1000  |
| testscript | 56561 | /usr/bin/dash  | /bin/sh /tmp/testscript 3000 | 0     |
+-----+-----+-----+-----+-----+
osquery> |

18 root      rt  0      0      0 S  0.0  0.0  0:00.06 migrati+

root@k8s-cp:~# jobs
root@k8s-cp:~# /tmp/testscript 3000 &
[1] 55911
root@k8s-cp:~# kill testscript
-bash: kill: `testscript': not a pid or valid job spec
root@k8s-cp:~# kill %1
[1]+  Terminated                  /tmp/testscript 3000
root@k8s-cp:~# jobs
root@k8s-cp:~# /tmp/testscript 3000
-

```

copying sleep to the /tmp folder and running the copy didn't work, so I introduced a script wrapper; making a script that calls sleep but the script is in temp, which cause the issue that now the path is /bin/sh and not/tmp/, while cmdline is in /tmp/

Listening ports:

```
osquery> SELECT pid, address, port, protocol, family FROM listening_ports WHERE port>0 ORDER BY port;
```

pid	address	port	protocol	family
48071	0.0.0.0	22	6	2
48071	::	22	6	10
618	127.0.0.53	53	6	2
618	127.0.0.54	53	6	2
618	127.0.0.54	53	17	2
618	127.0.0.53	53	17	2
35263	::	53	6	10
35263	::	53	17	10
782	::	58	255	10
782	192.168.94.128	68	17	2
1211	127.0.0.1	323	17	2
1211	:::1	323	17	10
33531	127.0.0.1	1338	6	2
-1	0.0.0.0	4789	17	2
35263	::	8080	6	10
35263	::	8181	6	10
35544	::	9094	6	10
34579	127.0.0.1	9099	6	2
35263	::	9153	6	10
33717	127.0.0.1	10248	6	2
33717	127.0.0.1	10249	6	2
33717	::	10250	6	10
33717	127.0.0.1	10256	6	2
33717	::	10257	6	10
33717	::	10259	6	10
33717	::	16443	6	10
1131	127.0.0.1	19001	6	2
33111	::	25000	6	10
33531	127.0.0.1	33821	6	2

```
osquery> |
```

→ filtered by ports>0 for more relevant info

⇒ Process with PID 782 has an active network connections to a remote port: 68

⇒ This listening_ports table is for local ports actively listening, so to find what we need we opt for the process_open_socket table:

```
osquery> select * from process_open_sockets WHERE remote_port>0 ORDER BY local_port;
```

pid	fd	socket	family	protocol	local_address	remote_address	local_port	remote_port	path	state	net_namespace
48195	9	212802	2	6	192.168.94.128	192.168.94.1	22	1107		ESTABLISHED	4026531833
-1	-1	0	10	6	::ffff:10.1.62.137	::ffff:192.168.94.128	8080	54588		TIME_WAIT	4026532732
-1	-1	0	10	6	::ffff:10.1.62.137	::ffff:192.168.94.128	8080	60538		TIME_WAIT	4026532732
-1	-1	0	10	6	::ffff:10.1.62.137	::ffff:192.168.94.128	8080	40312		TIME_WAIT	4026532732
-1	-1	0	10	6	::ffff:10.1.62.137	::ffff:192.168.94.128	8080	46322		TIME_WAIT	4026532732
-1	-1	0	10	6	::ffff:10.1.62.137	::ffff:192.168.94.128	8080	54864		TIME_WAIT	4026532732
-1	-1	0	10	6	::ffff:10.1.62.137	::ffff:192.168.94.128	8181	51256		TIME_WAIT	4026532732
-1	-1	0	10	6	::ffff:10.1.62.137	::ffff:192.168.94.128	8181	34026		TIME_WAIT	4026532732

Which shows the processes with active network connections to a remote port; in our case 48195

```
osquery> select pid, name, path, cmdline, state, uid from processes where pid=48195;
```

pid	name	path	cmdline	state	uid
48195	sshd-session	/usr/lib/openssh/sshd-session	sshd-session: amin@pts/1	S	1000

```
osquery> |
```

By querying the process table to find the process with that same PID, it turns out it is ssh.

If I am sure no admins should currently be connected through ssh, then this could be evidence for a malicious activity

FILE

```
osquery> SELECT path, filename, size, mode, uid, gid, mtime FROM file WHERE path = '/tmp/testfile.txt';
```

path	filename	size	mode	uid	gid	mtime
/tmp/testfile.txt	testfile.txt	12	0644	0	0	1789419922

Hash:

```
root@k8s-cp:~# sha256sum /tmp/testfile.txt
621dc6410401957fd620cb46a8f29165c24bf399a5d6726dfe080b15f8543e1d /tmp/testfile.txt
root@k8s-cp:~# _
```

V. CORRELATING OBSERVATIONS: FROM TABLES TO SECURITY CONTEXT

```
amin@k8s-cp:~$ osqueryi 'select lp.port, lp.address, lp.protocol, lp.pid, p.name, p.cmdline, p.uid from processes as p join listening_ports as lp ORDER BY lp.port;' | head -n 20
```

port	address	protocol	pid	name	cmdline	uid
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	
0	0	-1	systemd	/usr/lib/systemd/systemd --switched-root --system --deserialize=52	0	

```
osquery> select pid, port, address, path from listening_ports where port>0 and (address not like ':::%' and address not like '127.0.%') order by port;
```

pid	port	address	path
4865	22	0.0.0.0	
812	68	192.168.94.128	
-1	4789	0.0.0.0	
35096	8088	0.0.0.0	

```
osquery> select lp.pid, lp.port, lp.address, p.cmdline, p.path, p.uid from processes as p join listening_ports as lp where lp.pid=p.pid and lp.port>0 and (lp.address not like ':::%' and lp.address not like '127.0.%') order by lp.port;
```

pid	port	address	cmdline	path	uid
4865	22	0.0.0.0	sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups	/usr/sbin/sshd	0
812	68	192.168.94.128	/usr/lib/systemd/systemd-networkd	/usr/lib/systemd/systemd-networkd	998
35096	8088	0.0.0.0	python3 -m http.server 8088 --directory /tmp	/usr/bin/python3.14	1000

```
amin@k8s-cp:~$ ps aux | grep lab-updater
amin      49885  0.0  0.0  4260  2428 pts/0    S+   09:06   0:00 grep --color=auto lab-updater
```

```

amin@k8s-cp:~$ pkill -f '/tmp/.lab-updater.sh'
amin@k8s-cp:~$ pkill -f 'http.server 8088'
amin@k8s-cp:~$ rm -f /tmp/.lab-updater.sh /tmp/lab-http.log /tmp/testfile.txt
amin@k8s-cp:~$ sh -c 'echo short-lived-osquery-demo > /tmp/short-lived.txt'
amin@k8s-cp:~$ sudo osqueryi SELECT pid, name, path, cmdline FROM processes WHERE cmdline LIKE '%short-livedosquery-demo%';
[sudo: authenticate] Password:
Error: incomplete input
amin@k8s-cp:~$ sudo osqueryi SELECT pid, name, path, cmdline FROM processes WHERE cmdline LIKE '%short-livedosquery-demo%';
Error: incomplete input
amin@k8s-cp:~$ sudo osqueryi
Using a virtual database. Need help, type '.help'
osquery> SELECT pid, name, path, cmdline FROM processes WHERE cmdline LIKE '%short-livedosquery-demo%';
osquery> |

```

Configuring event-based telemetry:

Disabling auditd which conflicts with Osquery's

```

amin@k8s-cp:~$ sudo systemctl stop auditd
[sudo: authenticate] Password:
Failed to stop auditd.service: Unit auditd.service not loaded.
amin@k8s-cp:~$ sudo systemctl disable auditd
Failed to disable unit: Unit auditd.service does not exist

```

Configuring osquery:

```

amin@k8s-cp:~$ cat /etc/osquery/osquery.conf
{
  "options": {
    "disable_events": "false",
    "disable_audit": "false",
    "audit_allow_sockets": "true",
    "audit_allow_process_events": "true"
  },
  "schedule": {
    "query_process_events": {
      "query": "SELECT pid, path, cmdline, uid, gid, parent, time FROM process_events;",
      "interval": 10
    }
  }
}

```

Enabling osqueryd

```

amin@k8s-cp:~$ sudo systemctl restart osqueryd
amin@k8s-cp:~$ sudo systemctl status osqueryd
• osqueryd.service - osquery daemon
   Loaded: loaded (/usr/lib/systemd/system/osqueryd.service; disabled; preset: enabled)
   Active: active (running) since Tue 2026-09-15 09:21:57 UTC; 10s ago
   Invocation: f1fc049ec3044f039ff3ddf8c0e8e3de
   Process: 72284 ExecStartPre=/bin/sh -c if [ ! -f $FLAG_FILE ]; then touch $FLAG_FILE; fi (code=exited, status=0/SUCCESS)
   Process: 72287 ExecStartPre=/bin/sh -c if [ -f $LOCAL_PIDFILE ]; then mv $LOCAL_PIDFILE $PIDFILE; fi (code=exited, status=0/SUCCESS)
   Main PID: 72289 (osqueryd)
     Tasks: 17 (limit: 3850)
    Memory: 24.9M (peak: 24.9M)
       CPU: 64ms
   CGroup: /system.slice/osqueryd.service
           └─72289 /opt/osquery/bin/osqueryd --flagfile /etc/osquery/osquery.flags --config_path /etc/osquery/osquery.conf
             └─72295 /opt/osquery/bin/osqueryd

```

30. Additional engineering costs, could be the fact that the osqueryd should be always running before an event happens, otherwise it wouldn't be detected and also all events should be stored in the db, which consumes more storage.